PROJECT REPORT

ON

SAI DEEPAK FILM BOOKING SYSTEM

A Full-Featured Cinema Ticket Booking Web Application

Submitted by

Sai Deepak

Department of Computer Application

ABSTRACT

The Sai Deepak Film Booking System is a full-featured cinema ticket reservation web application developed using HTML5, CSS3, JavaScript, and PHP with a MySQL database backend. The system enables users to browse currently showing films, select preferred dates and showtimes, choose seats interactively from a visual seat map, and complete bookings through a streamlined three-step workflow. The application supports user authentication (registration and login), booking history management, and booking cancellation. It is designed to function both as a standalone front-end application (using localStorage as a fallback) and as a complete full-stack system when deployed on a PHP/MySQL server. The project demonstrates modern web development principles including responsive design, API-driven architecture, and graceful degradation.

TABLE OF CONTENTS

Chapter / Title	Page No.
1. Introduction	4
2. System Requirements	5
3. System Architecture	6
4. Modules Description	7
5. Database Design	9
6. File Structure	10
7. User Interface Design	11
8. Functional Description	13
9. Technology Stack	14
10. Testing	15
11. Limitations and Future Enhancements	16
12. Conclusion	17
13. References	18

1. INTRODUCTION

1.1 Background

The rapid growth of digital technology has significantly transformed the way entertainment services are accessed and consumed. Cinema ticket booking, once limited to physical box offices, has evolved into an online experience where users can plan, select, and confirm their movie experience from any device. Online booking systems improve customer convenience, reduce waiting times, and provide theatre management with automated record-keeping capabilities.

The Sai Deepak Film Booking System is designed to replicate and simplify this modern cinema booking experience. Built entirely with standard web technologies, it provides a fully functional booking workflow without requiring complex third-party frameworks.

1.2 Objectives

The primary objectives of this project are as follows:

- To develop a responsive and visually appealing cinema booking interface.
- To implement a multi-step booking workflow covering movie selection, seat selection, and payment confirmation.
- To provide user authentication including registration and login functionality.
- To integrate a PHP backend API with MySQL for persistent data storage.
- To ensure the system degrades gracefully to browser localStorage when a server backend is unavailable.
- To support booking management, including the ability to view and cancel previous bookings.

1.3 Scope

The scope of this project covers the development of a web-based cinema booking system that operates on standard web browsers without requiring additional plugins or installations. The system handles a catalogue of eight films across four genres. It supports up to eight seat selections per booking session and provides a seven-day rolling

date picker for showtime selection. The backend API is developed in PHP and communicates with a MySQL database.

2. SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 or equivalent (1.5 GHz+)
RAM	2 GB
Storage	100 MB free disk space
Network	Broadband internet connection (for full-stack deployment)
Display	1024 x 768 resolution or higher

2.2 Software Requirements

Software	Version / Details
Operating System	Windows 7 / macOS 10.12 / Linux (any modern distribution)
Web Browser	Google Chrome 90+, Mozilla Firefox 85+, Microsoft Edge 90+
Web Server	Apache 2.4+ or Nginx 1.18+ (for PHP backend)
PHP	Version 7.4 or higher
MySQL	Version 5.7 or higher
Node.js	Not required for deployment; used for development only
Text Editor / IDE	Visual Studio Code, Sublime Text, or any equivalent

2.3 Development Environment

The project was developed and tested using XAMPP (Apache + PHP + MySQL) on a local development machine. The frontend files were served directly from the htdocs directory. The MySQL database was managed using phpMyAdmin. For front-end only testing, no server was required and all files were opened directly in a web browser.

3. SYSTEM ARCHITECTURE

3.1 Overview

The Sai Deepak Film Booking System follows a classic three-tier web application architecture consisting of a Presentation Layer, a Business Logic Layer, and a Data Layer. The frontend is served as static HTML/CSS/JavaScript files, while the backend is a single PHP file (api.php) that handles all server-side operations and communicates with the MySQL database.

3.2 Architecture Diagram

The system architecture is structured as follows. The Client Layer consists of the web browser rendering index.html and booking.html along with their associated stylesheets and scripts. The Application Layer comprises the PHP backend file api.php which processes HTTP requests, performs authentication, and executes database queries. The Data Layer is the MySQL database named sai_deepak which stores user accounts, movie data, showtimes, and booking records.

The following diagram illustrates the three-tier architecture of the system. [Architecture diagram can be drawn at https://app.diagrams.net/]

[Insert Architecture Diagram Here — export PNG from app.diagrams.net]

Figure 1: Three-Tier System Architecture

3.3 Data Flow

When a user accesses the application, the browser loads the HTML pages and executes JavaScript. When the user performs an action such as registering, logging in, browsing movies, or submitting a booking, the JavaScript sends an asynchronous HTTP request (fetch API) to api.php with the relevant action parameter. The PHP script processes the request, queries or updates the MySQL database accordingly, and returns a JSON response. The JavaScript then updates the DOM to reflect the result. If the PHP backend is unavailable (USE_PHP_API is set to false), all operations fall back to browser localStorage.

4. MODULES DESCRIPTION

The system is divided into the following major functional modules:

4.1 User Authentication Module

This module handles all aspects of user identity management. It provides two sub-functions: user registration and user login.

Registration: A new user provides their full name, email address, and password. When the PHP API is active, the password is hashed using bcrypt via PHP's password_hash() function before storage. When operating in offline mode, credentials are stored directly in localStorage.

Login: Existing users authenticate by supplying their email and password. The PHP backend verifies the password against the stored hash using password_verify(). Upon successful authentication, user details are stored in localStorage under the key cvUser for session persistence.

Session management is handled client-side by reading the stored user object from localStorage. The user's name is displayed in the navigation bar upon login.

4.2 Movie Browsing Module

The movie browsing module presents a grid of available films on the home page (index.html). Each movie card displays the film title, genre, star rating, duration, and price per seat. A hero section at the top of the page highlights a featured film (Stellar Void) with an extended description and a direct booking link.

Genre Filtering: Users can filter the displayed films by genre using filter buttons (All, Action, Drama, Sci-Fi, Horror). Filtering is performed client-side without any additional server requests. When the PHP API is enabled, the movie list is loaded from the database. Otherwise, the static MOVIES array defined in app.js serves as the data source.

4.3 Booking Workflow Module

The booking process is structured as a three-step wizard implemented in booking.html and booking.js:

Step 1 — Film and Showtime Selection

The user selects a film from the movie selection grid. Upon selection, a showtime section appears offering a rolling seven-day date picker and five fixed showtime options. The user must select both a date and a showtime before proceeding. The Continue button remains disabled until all required selections are made.

Step 2 — Seat Selection

A visual seat map is rendered using a grid of seven rows (A through G) and ten columns. Pre-occupied seats (defined in the TAKEN constant) are displayed as unavailable. Users can select up to eight seats per booking. Selected seats are highlighted visually and their identifiers (e.g., A3, C7) are displayed below the map. The Continue button is enabled only when at least one seat is selected.

Step 3 — Confirmation and Payment

An order summary card displays all booking details including the selected film, date, showtime, seat identifiers, per-seat price, and total amount payable. The user fills in their name, email, and a demonstration card number with expiry and CVV. Basic validation is applied to all payment fields. On submission, the booking is sent to the PHP API. If the API call fails, the booking is saved to localStorage as a fallback.

4.4 My Bookings Module

Logged-in users can view their booking history on the home page. When connected to the PHP backend, bookings are retrieved by email address. Each booking item displays the film name, date, showtime, seat list, and booking status (Confirmed or Cancelled). A Cancel button is available for active bookings, which sends a cancellation request to the backend API.

4.5 PHP Backend API Module

The api.php file serves as the sole server-side component. It accepts requests via query parameters (action=<action_name>) and returns JSON responses. The supported actions are:

Action	Method	Description
register	POST	Creates a new user account with bcrypt password hashing
login	POST	Authenticates a user and returns user details
movies	GET	Returns the full list of active movies from the database
booking	POST	Saves a new booking record to the database
bookings	GET	Returns all bookings associated with a given email address
cancel	POST	Updates a booking status to cancelled

5. DATABASE DESIGN

5.1 Entity Relationship Overview

The database sai_deepak consists of four tables: users, movies, showtimes, and bookings. The bookings table references users and movies through foreign key relationships. A user can have multiple bookings. Each movie can have multiple showtimes.

The following entity relationship diagram illustrates the table relationships. [ER diagram can be created at https://app.diagrams.net/]

[Insert ER Diagram Here — export PNG from app.diagrams.net]

Figure 2: Entity Relationship Diagram

5.2 Table Descriptions

users Table

Field	Type	Description
id	INT UNSIGNED (PK, AUTO_INCREMENT)	Unique user identifier
name	VARCHAR(120) NOT NULL	User's full name
email	VARCHAR(180) NOT NULL UNIQUE	User's email address (used as login)
password	VARCHAR(255) NOT NULL	bcrypt-hashed password
created_at	DATETIME DEFAULT CURRENT_TIMESTAMP	Account creation timestamp

movies Table

Field	Type	Description
id	VARCHAR(80) (PK)	Slug-style unique movie identifier
title	VARCHAR(150) NOT NULL	Full movie title
genre	VARCHAR(50) NOT NULL	Genre category
rating	DECIMAL(3,1)	Star rating out of 10
duration	VARCHAR(30)	Film duration string (e.g., 2h 28m)
price	DECIMAL(6,2)	Per-seat ticket price in Rupees
emoji	VARCHAR(10)	Display emoji for the movie card
description	TEXT	Short film synopsis
active	TINYINT(1) DEFAULT 1	Soft delete flag

showtimes Table

Field	Type	Description
id	INT UNSIGNED (PK, AUTO_INCREMENT)	Unique showtime identifier
movie_id	VARCHAR(80) (FK)	References movies.id

Field	Type	Description
show_time	VARCHAR(20) NOT NULL	Showtime string (e.g., 10:30 AM)

bookings Table

Field	Type	Description
id	INT UNSIGNED (PK, AUTO_INCREMENT)	Unique booking identifier
user_id	INT UNSIGNED (FK, nullable)	References users.id
movie_id	VARCHAR(80) NOT NULL	References movies.id
movie_title	VARCHAR(150) NOT NULL	Denormalized movie title for display
date	DATE NOT NULL	Selected viewing date
showtime	VARCHAR(20) NOT NULL	Selected showtime
seats	TEXT NOT NULL	Comma-separated list of seat identifiers
name	VARCHAR(150) NOT NULL	Booker's full name
email	VARCHAR(180) NOT NULL	Booker's email address
total	DECIMAL(8,2) NOT NULL	Total amount paid
status	ENUM('confirmed','cancelled')	Booking status
created_at	DATETIME DEFAULT CURRENT_TIMESTAMP	Booking creation timestamp

6. FILE STRUCTURE

The project consists of six core files organized in a single directory. The table below describes each file and its purpose:

File	Type	Description
index.html	HTML	Main landing page displaying the hero section and movie grid
booking.html	HTML	Three-step booking wizard page

File	Type	Description
style.css	CSS	Complete stylesheet for the cinematic dark-theme UI
app.js	JavaScript	Core application logic: movie data, authentication, API wrapper, modals, toast notifications
booking.js	JavaScript	Booking-specific logic: seat map, step navigation, date picker, form handling, submission
api.php	PHP	Server-side REST API handling user authentication, movie retrieval, booking creation, and cancellation
sai_deepak.sql	SQL	Database schema and seed data for tables: users, movies, showtimes, bookings
README.md	Markdown	Developer documentation covering setup, configuration, and customization instructions

The project follows a flat file structure, which simplifies deployment. All HTML pages reference the shared stylesheet (style.css) and the core application script (app.js). The booking page additionally loads booking.js. The PHP API file is accessed via HTTP requests from the JavaScript code and does not need to be referenced in HTML.

7. USER INTERFACE DESIGN

7.1 Design Philosophy

The user interface follows a cinematic dark-theme aesthetic inspired by premium movie ticketing platforms. The colour palette is built around deep blacks, charcoal greys, and gold accents, evoking the atmosphere of a luxury cinema. The Bebas Neue display typeface is used for headings and titles, while Barlow is used for body text, providing a contrast between bold editorial typography and clean readable prose.

7.2 Home Page (index.html)

The home page is structured into three distinct sections. The Navigation Bar at the top contains the brand logo, navigation links to the main pages, and a user authentication control showing the logged-in user's name or a Sign In button. The Hero Section

presents the featured film (Stellar Void) with a large title, synopsis, metadata, and a Book Now call-to-action button. The background uses a CSS grain overlay to simulate film texture. The All Screenings Section displays all available films in a responsive grid. Each card includes an emoji poster, title, rating, duration, price, and genre tag. Hovering over a card reveals a Book overlay button.

7.3 Booking Page (booking.html)

The booking page presents a three-step wizard with a visual step indicator at the top. Step indicators are numbered 01, 02, and 03, and are connected by horizontal lines. The active step is visually highlighted while completed steps are marked as done.

Step 1 contains a movie selection grid (identical in style to the home grid), a date pill selector showing the next seven days, and time slot pills showing available showtimes with remaining seat counts. Step 2 renders the interactive seat map with row and column labels, a screen label at the top, and a legend explaining seat states. Step 3 presents an order summary card and payment input fields styled to resemble a credit card form.

7.4 Modals

Login and registration are handled through overlay modals. The modals appear as centred cards with a dark semi-transparent backdrop. They include form validation and a cross button to close them. Switching between login and registration modals is supported without closing and reopening.

7.5 Toast Notifications

A toast notification system provides brief feedback messages at the bottom of the screen. Toast messages appear for events such as successful login, booking confirmation, error validation, and cancellation. They auto-dismiss after 3.5 seconds and support success and error styling variants.

7.6 Responsive Design

The layout adapts to different screen sizes using CSS media queries. The movie grid transitions from a multi-column layout on desktop to a single-column layout on mobile.

Navigation links collapse on smaller screens. The seat map uses horizontal scrolling to remain usable on narrow viewports.

[Insert UI Wireframe / Screenshot Here — export PNG from app.diagrams.net]

Figure 3: Booking Page Wireframe — Three-Step Workflow

8. FUNCTIONAL DESCRIPTION

8.1 User Registration and Login

A visitor clicking the Sign In button opens the login modal. If they do not have an account, they can switch to the registration modal. Upon submitting the registration form, the handleRegister() function in app.js sends the name, email, and password to the PHP API's register action. The API validates that the email is unique, hashes the password, inserts the record, and returns the user object. The frontend stores this in localStorage and updates the navigation bar greeting.

For login, handleLogin() sends the email and password to the login action. The API fetches the user by email and verifies the password. On success, the user object is returned and stored locally.

8.2 Booking Submission

When the user reaches Step 3 and fills in the payment details, the submitBooking() function in booking.js gathers all selected values (movie, date, showtime, seats, name, email) and calls apiPost('booking', payload). The PHP API inserts a new record into the bookings table and returns the generated booking ID. The success modal is then displayed with the booking summary. If the API is unavailable, the payload is saved to localStorage under cvBookings.

8.3 Booking Cancellation

The cancelBooking(id) function calls the cancel action with the booking ID. The PHP API performs an UPDATE statement to set the booking status to cancelled. The booking list is re-rendered to reflect the updated status.

8.4 API Toggle

The constant USE_PHP_API at the top of app.js controls whether the system uses the PHP backend or operates entirely offline. When set to false, all data operations use localStorage and the static MOVIES array. This design allows the frontend to be tested and demonstrated without a server.

9. TECHNOLOGY STACK

Layer	Technology	Purpose
Markup	HTML5	Page structure and semantic content
Styling	CSS3	Dark-theme UI, animations, responsive layout
Client Logic	Vanilla JavaScript (ES6+)	DOM manipulation, API calls, state management
Server Logic	PHP 7.4+	REST API, authentication, database interaction
Database	MySQL 5.7+	Persistent storage of users, movies, and bookings
Typography	Google Fonts (Bebas Neue, Barlow)	Display and body typefaces
HTTP Client	Fetch API	Asynchronous communication with api.php
Storage Fallback	localStorage (Browser)	Offline data persistence

The project deliberately avoids front-end frameworks such as React or Vue.js, and back-end frameworks such as Laravel or Symfony. This decision keeps the deployment requirements minimal and ensures the code is accessible to developers at all skill levels. The entire front-end runs from static HTML files and does not require a build step.

10. TESTING

10.1 Testing Approach

The system was tested through manual functional testing. Each module was tested independently before end-to-end testing of the complete booking workflow. Testing was performed in both online mode (PHP + MySQL) and offline mode (localStorage fallback).

10.2 Test Cases

Test ID	Test Case	Expected Result	Status
TC-01	User registers with valid details	Account created, user logged in, toast displayed	Pass
TC-02	User registers with duplicate email	Error toast: Email already registered	Pass
TC-03	User logs in with valid credentials	User authenticated, name shown in navbar	Pass
TC-04	User logs in with invalid password	Error toast: Invalid credentials	Pass
TC-05	Browse movie grid	All 8 movies displayed with correct details	Pass
TC-06	Filter movies by genre	Only movies of selected genre shown	Pass
TC-07	Select movie in booking page	Showtime section appears	Pass
TC-08	Select date and time	Continue button enabled	Pass
TC-09	Select seats (up to 8)	Seats highlighted, display updated	Pass
TC-10	Attempt to select 9th seat	Error toast: Max 8 seats per booking	Pass
TC-11	Complete booking with valid payment data	Success modal shown with booking ID	Pass
TC-12	Submit booking without signing in	Login modal opens, booking paused	Pass
TC-13	View booking history	All bookings listed with correct details	Pass

Test ID	Test Case	Expected Result	Status
TC-14	Cancel a booking	Status changes to Cancelled in list	Pass
TC-15	Booking with PHP unavailable	Booking saved to localStorage, success modal shown	Pass

10.3 Browser Compatibility

The application was tested on the following browsers: Google Chrome 120, Mozilla Firefox 115, and Microsoft Edge 120. All features functioned correctly across these browsers. The Fetch API and localStorage are natively supported in all modern browsers and do not require polyfills.

11. LIMITATIONS AND FUTURE ENHANCEMENTS

11.1 Current Limitations

- The payment module is a demonstration only and does not integrate with any real payment gateway. Card numbers are not validated against actual card networks.
- Session management is handled entirely via localStorage and does not use secure server-side sessions or tokens. This approach is not suitable for production deployment without modification.
- Seat availability is partially static. The TAKEN constant in booking.js defines a fixed list of occupied seats rather than dynamically querying real-time seat occupancy from the database.
- The system does not send email confirmations to users after booking.
- There is no administrative interface for theatre managers to add movies, manage showtimes, or view booking reports.
- Password recovery (forgot password) functionality is not implemented.

11.2 Future Enhancements

1. Integration with a payment gateway such as Razorpay or PayPal to enable real transactions.

2. Implementation of real-time seat availability checks by querying booked seats per movie, date, and showtime from the database before rendering the seat map.
3. Addition of email confirmation and e-ticket generation using PHP Mailer.
4. Development of an administrative dashboard for movie and showtime management.
5. Introduction of user profile management including password change and booking preferences.
6. Progressive Web App (PWA) support to allow offline browsing and home screen installation.
7. Server-side session management using PHP sessions or JSON Web Tokens (JWT) for improved security.

12. CONCLUSION

The Sai Deepak Film Booking System successfully demonstrates the development of a complete, end-to-end cinema ticket reservation application using standard web technologies. The project covers all essential aspects of a real-world booking system including user authentication, movie browsing, interactive seat selection, payment capture, and booking management.

The application is designed with a dual-mode operation strategy: it functions fully offline using browser localStorage, making it easy to demonstrate and test without server infrastructure, and it upgrades seamlessly to a fully persistent experience when deployed on a PHP/MySQL server. This flexibility makes the project both practical as a learning exercise and deployable as a production prototype with relatively minor modifications.

The cinematic dark-theme user interface provides an engaging and professional experience. The three-step booking wizard guides users through the reservation process in a logical, intuitive manner. The project demonstrates competence in frontend development (HTML, CSS, JavaScript), backend development (PHP, MySQL), relational database design, REST API design, and responsive user interface construction.

In conclusion, this project meets all stated objectives and provides a solid foundation upon which real-world enhancements such as payment gateway integration, real-time seat management, and administrative tools could be built.

13. REFERENCES

8. Mozilla Developer Network (MDN). HTML5 Reference. Available at: https://developer.mozilla.org/en-US/docs/Web/HTML

9. Mozilla Developer Network (MDN). CSS3 Reference. Available at: https://developer.mozilla.org/en-US/docs/Web/CSS

10. Mozilla Developer Network (MDN). JavaScript Guide and Reference. Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript

11. PHP Group. PHP Manual — Version 7.4. Available at: https://www.php.net/manual/en/

12. MySQL Documentation. MySQL 5.7 Reference Manual. Available at: https://dev.mysql.com/doc/refman/5.7/en/

13. Google Fonts. Bebas Neue and Barlow Typefaces. Available at: https://fonts.google.com

14. diagrams.net. Free Online Diagram Software. Available at: https://app.diagrams.net/

15. XAMPP. Cross-platform Apache, MySQL, PHP, and Perl distribution. Available at: https://www.apachefriends.org/

THANK YOU